FCMA203 逻辑与计组 **Lab 1 Report** **Fall 2026**

Deadline of submission: 23:59:59, Sep. 20th (Sunday), 2026

Name : 邓淳天 .Stu. ID : SUAT25000185 .

Lab Date : Thu Sep 10 2026 . Lab Time : 13:30-14:55 .

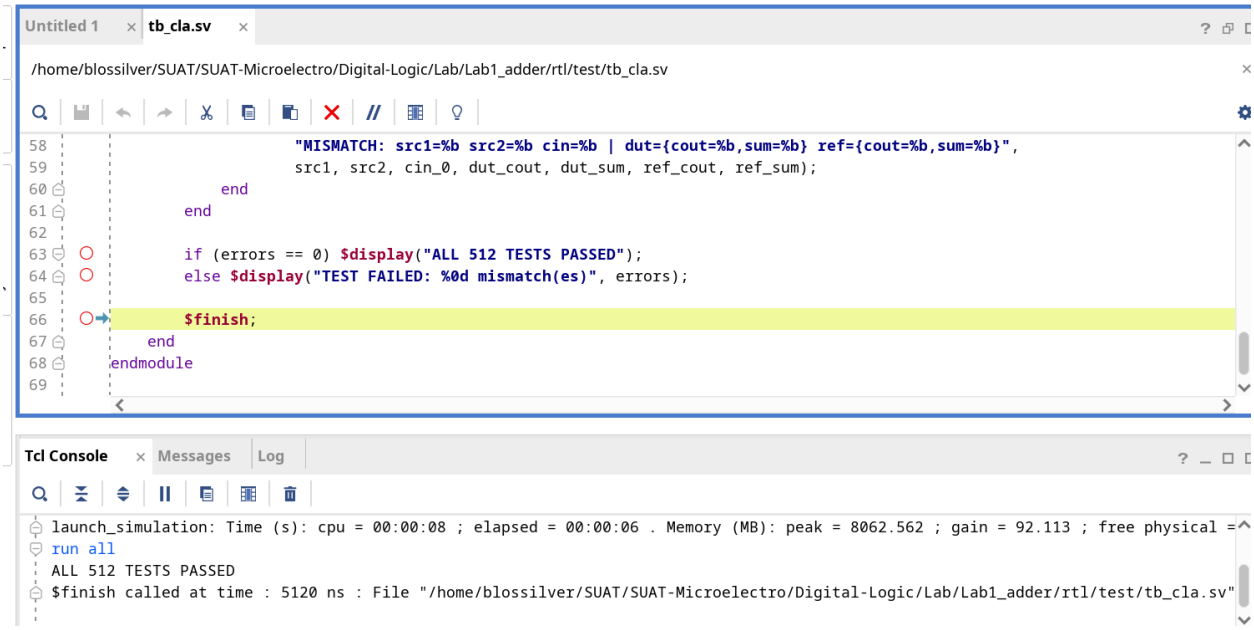
必做任务 2: 编写 4 位超前进位加法器的 Testbench

请写出你的具体测试思路。（可用流程图、框图等表示）

> 穷举全部输入组合以覆盖所有情况，把组合输入 dut 和 ref 中，并进行 difftest，若出现错误则通过内置函数$display 输出导致错误的输入组合。
>
> 参考模型 ref_ra 用一行 Verilog 行为描述实现正确加法： assign {cout, sum} = src1 + src2 + cin_0;

必做内容 3：仿真 4 位超前进位加法器

请将测试通过的截图粘贴在此。



选做内容：16 位层次化超前进位加法器

请描述你的代码实现思路、具体测试思路（可用流程图、框图等表示），并粘贴测试通过的截图。

实现思路：

1. 改造 4 位 cla，增加 Group Generate 和 Group Propagate 的输出
2. 实例化 4 个 4 位 cla 加法器，块 cla 间的进位信号同样通过 Group Generate 和 Group Propagate 信号，利用超前进位产生 cin

测试思路：

1. 沿用 ref_ra 作为参考模型 ref，以 cla_block 作为 dut，二者作 difftest
2. 先测试几个边界用例：
   - 全 0、全 1
   - 0xFFFF + 1、0xFFFF + 0xFFFF + 1（最长进位传播）
   - 0xAAAA + 0x5555 + 1（交替位）
   - 0x8000 + 0x8000（高位进位测试）
3. 接着随机生成 1w 组输入组合进行测试
4. 期间若有 dut != ref 则通过 $display 打印导致错误结果的输入组合

```
73        for (i = 0; i < NUM_RANDOM; i++) begin
74            check($urandom, $urandom, $urandom_range(1));
75        end
76        /* verilator lint_on WIDTHTRUNC */
77
78        if (errors == 0) $display("ALL %0d TESTS PASSED", NUM_RANDOM + 8);
79        else $display("TEST FAILED: %0d mismatch(es)", errors);
80
81        $finish;
82    end
83  endmodule
84
```

```
Tcl Console  ×  Messages  Log
ERROR: Could not open VCD file build/waves.fst for writing.
run all
ALL 10008 TESTS PASSED
$finish called at time : 100080 ns : File "/home/blossilver/SUAT/SUAT-Microelectro/Digital-Logic/Lab/Lab1_adder/rtl/test/tb_cla_b
```

Reflection: 基于你本次可能并不顺利的实验经历写下你的思考（300 字以内）：

你可以思考（但不限于）以下问题：

- 结果是否与你预期的一致？
- 在设计过程中是否遇到了一些问题或困难？你是如何解决它们的？原因是什么？
- 对于 Verilog 编码和仿真，有什么建议或技巧吗？
- ……

本次实验的结果与预期一致。设计过程遇到的主要困难是对于 testbench 的编写不熟悉。解决方法主要是靠阅读讲义提供的 testbench 示例，以及上网搜索 testbench 写法以实现自己需要的功能。

对于 verilog 编码和仿真，我的建议主要有：

1. 配置一个顺手熟悉的编辑器，利用好 verilog lsp 的实时静态语义分析功能，及时修改错误的地方。

2. 利用好仿真器的 lint 功能（如 verilator –lint-only），在完成编写，准备进行第一次测试时先跑一次 lint 检查，改掉仿真前就能发现的问题。

3. 测试时多利用内置函数及时发现 bug，如$assert $display。大致定位到问题后如果无法通过内置函数提供的信息解决才去阅读波形